Des agents IA aux données fiables

HydroDataLab
Une pratique concrète pour les analyses Python

20 octobre 2026 • Sources vérifiées le 18 octobre • Démonstration pédagogique, données publiques

Du besoin BI au résultat vérifié

Notre défi : détecter les mesures manquantes dans une journée de données.

Ce que vous saurez faire

Donner une tâche précise à l’agent.
Retrouver les règles qu’il utilise.
Vérifier le calcul qu’il propose.

Notre exemple Python

Données publiques Hydro-Québec.
Un traitement réutilisable.
Un indicateur de qualité visible.

> 45 minutes de présentation et de démos, puis vos questions.

Quatre séries, quatre sens différents

Série	Unité	Ce que l’on observe
Demande électrique au Québec	MW	Une puissance demandée
Niveau d’eau au réservoir Gouin	m	Une hauteur dans le référentiel source
Température minimale	°C	Un minimum sur la période source
Température maximale	°C	Un maximum sur la période source

Notre ETL : extraire les données, les transformer, puis charger ou exporter le résultat.
Un niveau en mètres ne donne pas une réserve en GWh.

Sources publiques HQ et contrat [S11, S12]. Instantané : 17 octobre 2026, 168 observations; récupéré le 18 octobre à 19:44 UTC.

Un modèle, des outils, un agent

Notion	Dans notre tâche Python
Modèle	Produit une proposition à partir des informations reçues.
Outils	Lisent un fichier, le modifient ou exécutent les tests.
Agent	Utilise le modèle et les outils pour avancer dans la tâche.

Lire → **Proposer** → **Tester** → **Corriger**

> Vous fixez l'objectif et les autorisations; vous examinez le résultat.

Fonctionnement simplifié d'un agent [S05, S06].

Git : garder une trace de ses essais

Mot rencontré	Ce qu'il signifie
Dépôt Git	Le projet et l'historique de ses versions enregistrées.
Commit	Un point enregistré dans cet historique.
Branche	Une suite de versions pour un essai, par exemple `demo-live`.
Worktree · facultatif	Un autre dossier du même dépôt, pour travailler sur une autre branche.

> Pour débuter, une branche d'essai suffit.

Démo 1 · donner une mission à l'agent

> « Repère le calcul des indicateurs et explique comment il est testé.
> Ne modifie aucun fichier. »

Repère dans la fenêtre Agents	À quoi il sert
Session	Garder une conversation consacrée à cette tâche.
Planification	Demander une démarche avec `/plan` avant de modifier.
Autorisations	Choisir quelles actions doivent être confirmées.

Quelles sources permettent de vérifier sa réponse?

Fenêtre Agents, modes et réglages [S01, S02, S03, S04, S16].

Des consignes qui survivent au chat

Une instruction permanente est une consigne conservée dans le projet pour les prochaines tâches.

Pour tout le projet

« Conserver les données brutes. »

`.github/copilot-instructions.md`

Pour les fichiers Python

« Une absence reste absente; zéro reste une valeur valide. »

`.github/instructions/python.instructions.md`

> L’agent reçoit la règle; les tests vérifient le comportement.

Instructions communes et ciblées [S03]. Leur découverte dépend du moteur d’exécution choisi.

Limiter une consigne aux fichiers utiles

`applyTo` indique les fichiers concernés.
`etl/**/*.py` signifie : fichiers Python dans `etl/` et ses sous-dossiers.

```
---
description: Qualité des transformations Python
applyTo: "etl/**/*.py"
---
Une absence reste absente. Zéro est une valeur valide.
```

> Exemple : le test doit compter `0`, mais ignorer une valeur absente.

Fichier Markdown .github/instructions/python.instructions.md [S03]. Exemple simplifié.

Réutiliser une recette ou un rôle

Une recette : une skill

Des étapes réutilisables pour accomplir une tâche précise.

Exemple : **valider un ETL**.
Lancer les tests, lire le rapport et vérifier les unités.

`validate-etl/SKILL.md`

Un agent personnalisé

Un rôle avec ses consignes et ses outils autorisés.

Exemple : **réviseur BI**.
Relire le calcul et signaler les défauts, sans modifier le code.

`reviseur-bi.agent.md`

> La recette décrit les étapes; le rôle définit la mission de revue.

Skills et agents personnalisés [S07, S08]. Exemples présents dans .github/.

Le contexte par paliers

Le contexte est l'information disponible pour accomplir la tâche.

1. Socle | 2. Composant | 3. Contrat | 4. Tâche | 5. Preuves

Ce qui reste vrai pour tout le dépôt

Python calcule les indicateurs. Les données brutes sont conservées. Chaque résultat garde son unité et sa source.

`.github/copilot-instructions.md`

Vue d'ensemble : socle → composant → contrat → tâche → preuves.
Choisir les détails utiles, sans joindre tous les fichiers à chaque demande.

Les tokens : lire et produire

Un token est un morceau de texte :
un mot, une partie de mot ou un signe.

Ce que le modèle reçoit

Votre demande et les consignes.
Les extraits de code ou de données.
Les résultats des outils.

Ce que le modèle produit

Sa réponse et le code proposé.
Du raisonnement facturé
selon le modèle utilisé.

> Pour notre ETL : le schéma, un petit échantillon représentatif
> et les tests donnent un contexte utile.

Contexte et facturation [S05, S06, S13]. Évaluer le coût de toutes les tentatives nécessaires.

Auto : un bon point de départ

Niveau	Priorité	Exemple BI
Efficiency	Coût	Expliquer une fonction courte
Balance	Coût, qualité et temps d’attente	Transformer une série avec tests
Intelligence	Qualité	Investiguer une anomalie difficile

> Auto choisit selon la tâche et les modèles autorisés.

Plans payants : remise Auto de 10 % sur le coût des modèles.
Le volume total dépend du travail effectué.

Documentation Auto consultée le 18 octobre 2026 [S14]. Politiques et abonnement restent déterminants.

Choisir et comparer sans deviner

Tâche Modifier un ETL testé Modèle GPT-6.1 Sol

Auto Balance, Sol ou Sonnet : vérifier le résultat avec les tests.

Simulation : **0.060 $ US** = 6.0 crédits

– 1 tentative(s) +

Hypothèse par tentative : 20 000 tokens d’entrée + 2 000 de sortie, sans cache ni remise Auto. Ni mesure réelle, ni classement de qualité. Tarifs standards du 18 octobre 2026.

Tarifs Copilot vérifiés le 18 octobre 2026 [S13]. Recommandations pédagogiques à évaluer sur vos tâches; ce n’est pas un benchmark.

Auto et HydraFusion : deux mécanismes

Auto choisit un modèle

Il sélectionne un modèle selon la tâche, sa disponibilité et les règles du compte.

HydraFusion en combine

Plusieurs modèles contribuent à une réponse, avec des étapes de vérification et de correction.

> HydraFusion Max consacre plus d’effort aux tâches difficiles.
> Il faut mesurer le coût total du travail effectué.

HydraFusion et Max : version expérimentale (« research preview »), 14 octobre 2026 [S14, S15]. Disponibilité selon le compte.

Démo 2 · suivre le trajet des données

Données HQ → **Calcul Python** → **Tableau de bord**

Élément préparé	Son rôle
API .NET	Transmettre les commandes et les résultats à l'interface Vue.
Aspire	Démarrer les composants et regrouper leurs journaux.
Image Docker	Emballer le code et ses dépendances pour l'exécuter en conteneur.

> D'où vient ce résultat? Et comment reprendre un traitement en erreur?

Architecture HydroDataLab [S12]. Le calcul métier reste dans le même code Python.

Une spécification avant le code

Une spécification décrit le résultat attendu et ses règles.
Ici, la **couverture** est la part des créneaux attendus qui ont une mesure valide.

Exemple sur une série horaire	Critère de réussite
24 mesures valides sur 24 attendues	100 %
23 mesures valides sur 24 attendues	95,83 %
Valeur zéro, doublon ou absence	Zéro compte; doublon et absence n’ajoutent rien.
Changement d’heure	23 ou 25 heures attendues selon la journée.

> Ces exemples deviennent des tests exécutables avant de demander le code.

Cas d’acceptation pédagogiques, pas des mesures HQ [S12, S16]. Journée locale : America/Toronto.

Démo 3 · détecter une mesure absente

24 mesures horaires attendues.
23 mesures numériques valides.

23 ÷ 24 = 95,83 %

> L’agent ajoute le calcul en Python.
> Nous vérifions les tests et l’indicateur affiché.

Scénario pédagogique : une mesure est retirée en mémoire. Les données brutes restent intactes.

Votre prochaine tâche Python

Choisissez un calcul dont vous savez vérifier le résultat.
Écrivez un exemple attendu, puis confiez-le à l'agent.

Apprendre pas à pas

Microsoft Learn : agents
Guides pratiques VS Code

Exemples et échanges

Awesome Copilot
GitHub Community
Spec Kit : organiser les spécifications

> Quelle règle de vos analyses aimeriez-vous rendre vérifiable?

Aide-mémoire et exercice participant fournis. Ressources [S06, S16, S17, S18].

Annexe · les réglages utiles

Réglage exact	Usage et réserve
`sessions.editor.wordWrap`	Retour à la ligne; expérimental
`sessions.layout.singlePaneDetailPanel`	Panneau intégré; recharger
`chat.useCustomizationsInParentRepositories`	Depuis un sous-dossier; défaut false
`github.copilot.chat.cli.customAgents.enabled`	Agents Copilot; vérifier la valeur
`chat.tools.compressOutput.enabled`	Sorties compressées; Preview
`chat.useNestedAgentsMdFiles`	Local; expérimental, défaut false

> Les anciens `chat.*FilesLocations` ne configurent pas tous le nouveau moteur Copilot.

Référence VS Code, 14 octobre 2026 [S04]. Vérifier portée et disponibilité dans la version installée.

Annexe · modèles et tarifs Copilot

Modèle	Entrée	Cache lu	Cache écrit	Sortie
GPT-6 Luna	0,10	0,01	0,125	0,50
Claude Haiku 5.5	0,10	0,01	0,125	0,50
GPT-6.1 Sol	2,00	0,10	2,50	10,00
Claude Sonnet 5.5	2,00	0,10	2,50	10,00
Claude Opus 5.5	4,00	0,20	5,00	20,00

$ US / million de tokens, tarifs standards. 1 AI credit = 0,01 $ US.
Long contexte : Haiku > 100k; Luna et Sol > 272k → grille distincte.

Tarifs GitHub Copilot au 18 octobre 2026 [S13]. Pas une garantie de disponibilité HQ ni un classement de qualité.

Annexe · une skill de validation

```
---
name: validate-etl
description: Vérifier une transformation Python et son rapport.
disable-model-invocation: true
---
```

1. Lire les critères et le contrat concernés.
2. Exécuter les tests ciblés et les contrôles prévus.
3. Vérifier unités, couverture et provenance.
4. Résumer les preuves et ce qui reste incertain.

Exemple pédagogique; commandes à aligner sur le dépôt. Format et déclenchement [S07, S09].

Annexe · le contrat d'une observation

Dimension	Question à résoudre
Schéma et identité	Quelle série, station et instant unique?
Unité et sens	MW, m ou °C? Mesure, minimum, maximum?
Fréquence	15 minutes, horaire ou quotidien?
Temps	Instant UTC; journée America/Toronto?
Provenance	URL, licence, récupération et version brute?
Qualité	Null, doublon, invalidité et couverture attendue?

Contrat HydroDataLab [S12] et métadonnées publiques HQ [S11]. Préserver le sens de la source.

Annexe · reprendre sans improviser

00 socle → **01 instructions** → **02 extraction** → **03 BI** → **04 pilotage** → **05 Docker** → **06 spec** → **07 solution**

- Un tag Git est un repère nommé sur une version enregistrée.
- Conserver ses modifications avant de changer de version.
- Séparer les données d’exécution des fichiers de code.
- Reprendre avec la copie locale des données et les tests.

> PDF, solution préparée et Git pour reprendre.

Guide présentateur et procédure de reprise du projet. Les checkpoints VS Code n’annulent pas les effets externes des outils [S06].

Annexe · sources et limites

VS Code et GitHub

Personnalisations
Contexte
Auto
Tarifs Copilot
HydraFusion, 14 octobre

Données et pratique

Données ouvertes HQ
Microsoft Learn
Awesome Copilot
Spec Kit

> Sources consultées le 18 octobre 2026. Les fonctions disponibles dépendent de la version et du compte.

Registre complet : docs/SOURCES.md. Schémas pédagogiques; aucune capture VS Code inventée.